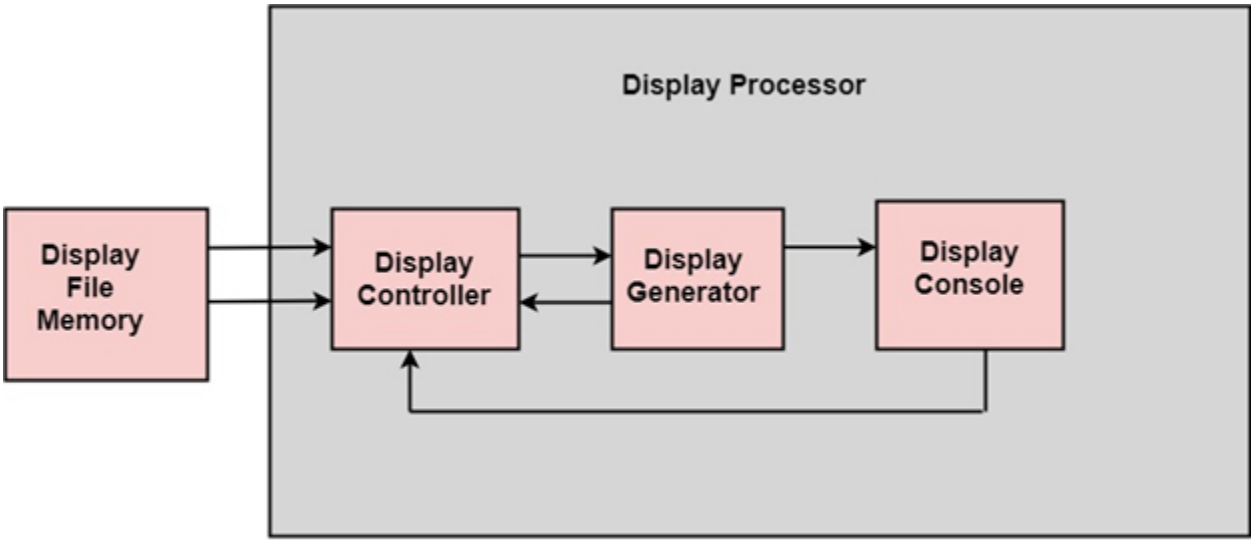
Display Processor:

It is interpreter or piece of hardware that converts display processor code into pictures. It is one of the four main parts of the display processor

Parts of Display Processor:

1. Display File Memory
2. Display Processor
3. Display Generator
4. Display Console



Block diagram of Display System

Display File Memory: It is used for generation of the picture. It is used for identification of graphic entities.

Display Controller:

1. It handles interrupt
2. It maintains timings
3. It is used for interpretation of instruction.

Display Generator:

1. It is used for the generation of character.
2. It is used for the generation of curves.

Display Console: It contains CRT, Light Pen, and Keyboard and deflection system.

The raster scan system is a combination of some processing units. It consists of the control processing unit (CPU) and a particular processor called a display controller. Display Controller controls the operation of the display device. It is also called a video controller.

Working: The video controller in the output circuitry generates the horizontal and vertical drive signals so that the monitor can sweep. Its beam across the screen during raster scans.

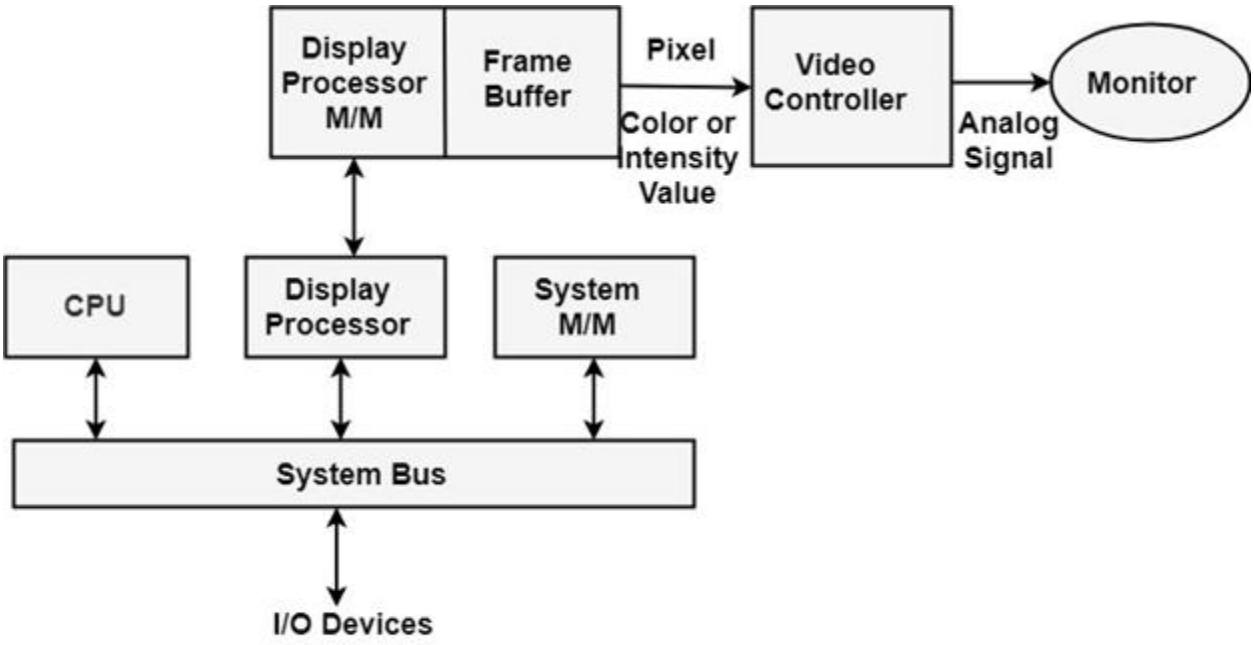


Fig: Architecture of a Raster Display System with a Display Processor

As fig showing that 2 registers (X register and Y register) are used to store the coordinate of the screen pixels. Assume that y values of the adjacent scan lines increased by 1 in an upward direction starting from 0 at the bottom of the screen to y_{max} at the top and along each scan line the screen pixel positions or x values are incremented by 1 from 0 at the leftmost position to x_{max} at the rightmost position.

The origin is at the lowest left corner of the screen as in a standard Cartesian coordinate system.

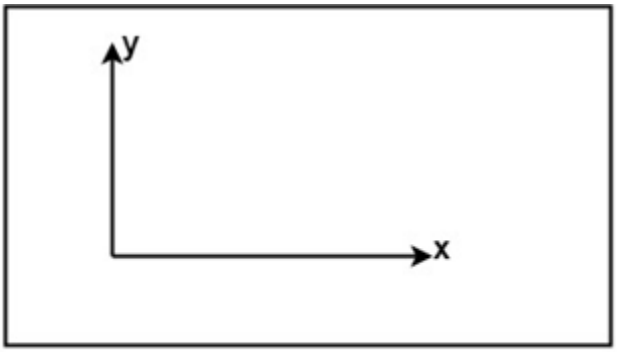


Fig: The origin of the coordinate system for identifying screen positions is usually specified in the lower-left corner.

At the start of a **Refresh Cycle**:

X register is set to 0 and y register is set to y_{max}. This (x, y') address is translated into a memory address of frame buffer where the color value for this pixel position is stored.

The controller receives this color value (a binary no) from the frame buffer, breaks it up into three parts and sends each element to a separate Digital-to-Analog Converter (DAC).

These voltages, in turn, controls the intensity of 3 e-beam that are focused at the (x, y) screen position by the horizontal and vertical drive signals.

This process is repeated for each pixel along the top scan line, each time incrementing the X register by Y.

As pixels on the first scan line are generated, the X register is incremented throughx_{max}.

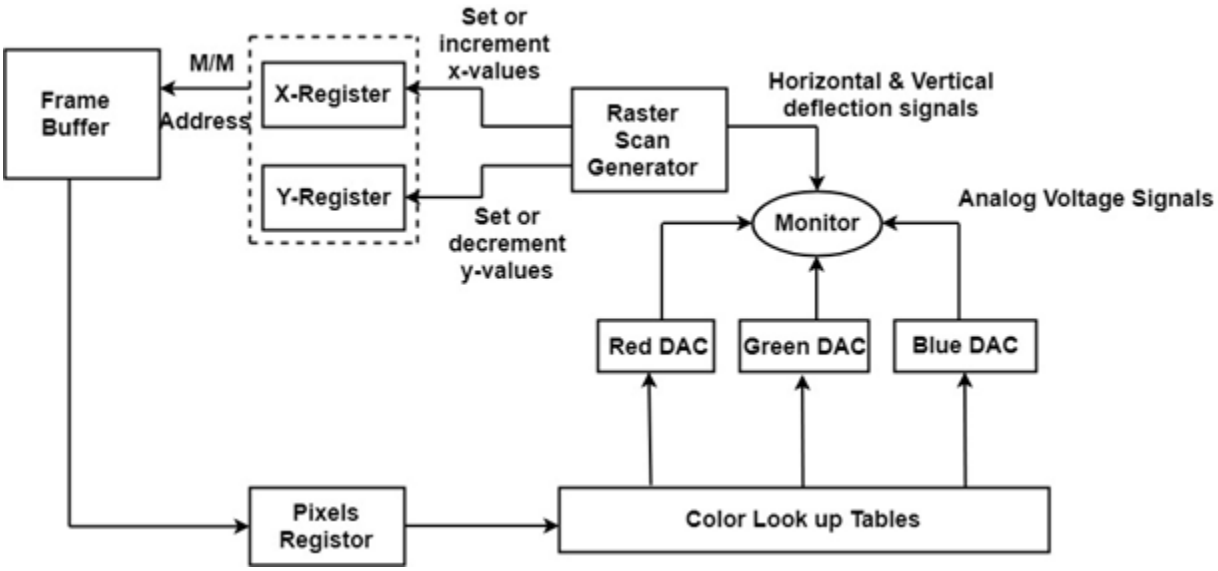
Then x register is reset to 0, and y register is decremented by 1 to access the next scan line.

Pixel along each scan line is then processed, and the procedure is repeated for each successive scan line units pixels on the last scan line (y=0) are generated.

For a display system employing a color look-up table frame buffer value is not directly used to control the CRT beam intensity.

It is used as an index to find the three pixel-color value from the look-up table. This lookup operation is done for each pixel on every display cycle.

As the time available to display or refresh a single pixel in the screen is too less, accessing the frame buffer every time for reading each pixel intensity value would consume more time what is allowed:



Multiple adjacent pixel values are fetched to the frame buffer in single access and stored in the register.

After every allowable time gap, the one-pixel value is shifted out from the register to control the warm intensity for that pixel.

The procedure is repeated with the next block of pixels,and so on, thus the whole group of pixels will be processed.

Display Devices:

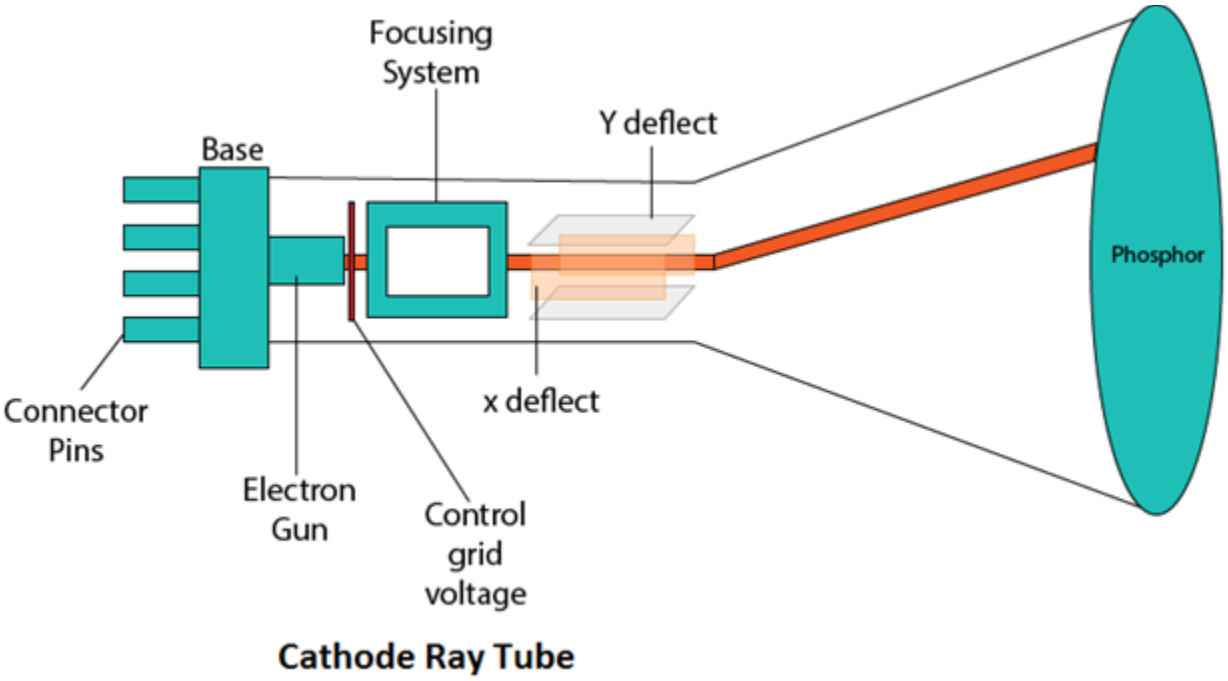
The most commonly used display device is a video monitor. The operation of most video monitors based on CRT (Cathode Ray Tube). The following display devices are used:

1. Refresh Cathode Ray Tube
2. Random Scan and Raster Scan
3. Color CRT Monitors
4. Direct View Storage Tubes
5. Flat Panel Display
6. Lookup Table

Cathode Ray Tube (CRT):

CRT stands for Cathode Ray Tube. CRT is a technology used in traditional computer monitors and televisions. The image on CRT display is created by firing electrons from the back of the tube of phosphorus located towards the front of the screen.

Once the electron heats the phosphorus, they light up, and they are projected on a screen. The color you view on the screen is produced by a blend of red, blue and green light.



Cathode Ray Tube

Components of CRT:

Main Components of CRT are:

1. Electron Gun: Electron gun consisting of a series of elements, primarily a heating filament (heater) and a cathode. The electron gun creates a source of electrons which are focused into a narrow beam directed at the face of the CRT.

2. Control Electrode: It is used to turn the electron beam on and off.

3. Focusing system: It is used to create a clear picture by focusing the electrons into a narrow beam.

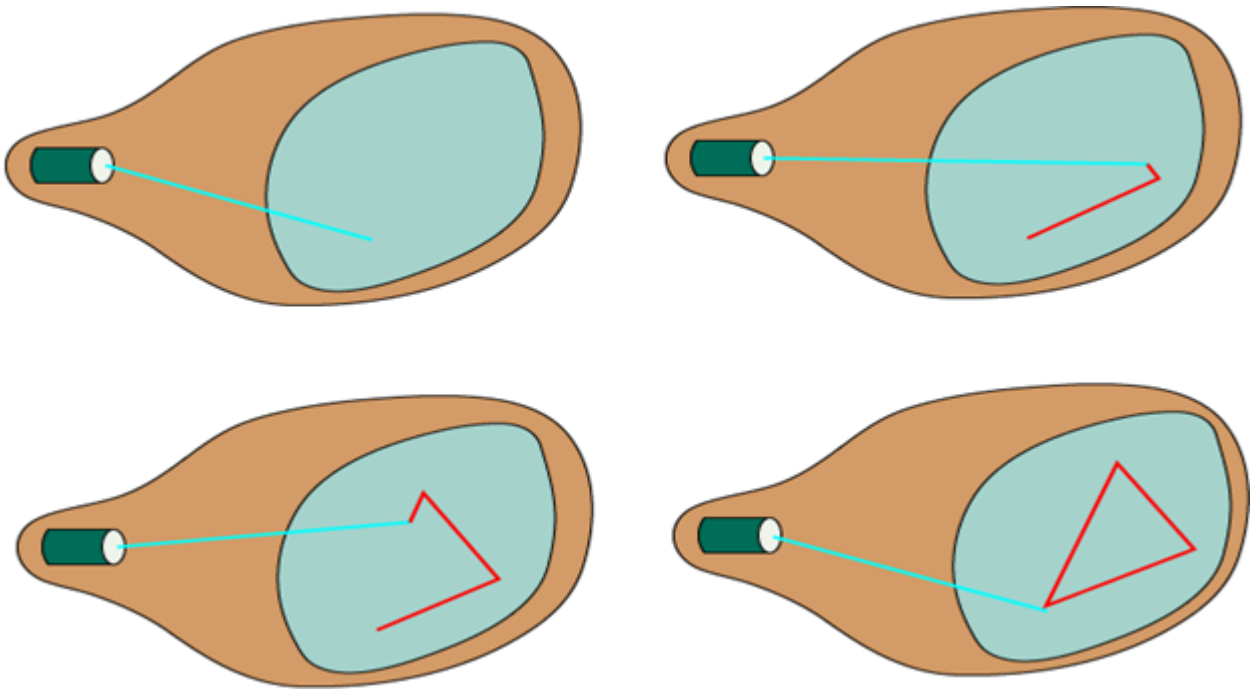
4. Deflection Yoke: It is used to control the direction of the electron beam. It creates an electric or magnetic field which will bend the electron beam as it passes through the area. In a conventional CRT, the yoke is linked to a sweep or scan generator. The deflection yoke which is connected to the sweep generator creates a fluctuating electric or magnetic potential.

5. Phosphorus-coated screen: The inside front surface of every CRT is coated with phosphors. Phosphors glow when a high-energy electron beam hits them. Phosphorescence is the term used to characterize the light given off by a phosphor after it has been exposed to an electron beam.

Random Scan and Raster Scan Display:

Random Scan Display:

Random Scan System uses an electron beam which operates like a pencil to create a line image on the CRT screen. The picture is constructed out of a sequence of straight-line segments. Each line segment is drawn on the screen by directing the beam to move from one point on the screen to the next, where its x & y coordinates define each point. After drawing the picture. The system cycles back to the first line and design all the lines of the image 30 to 60 time each second. The process is shown in fig:



Random-scan monitors are also known as vector displays or stroke-writing displays or calligraphic displays.

Advantages:

1. A CRT has the electron beam directed only to the parts of the screen where an image is to be drawn.
2. Produce smooth line drawings.

3. High Resolution

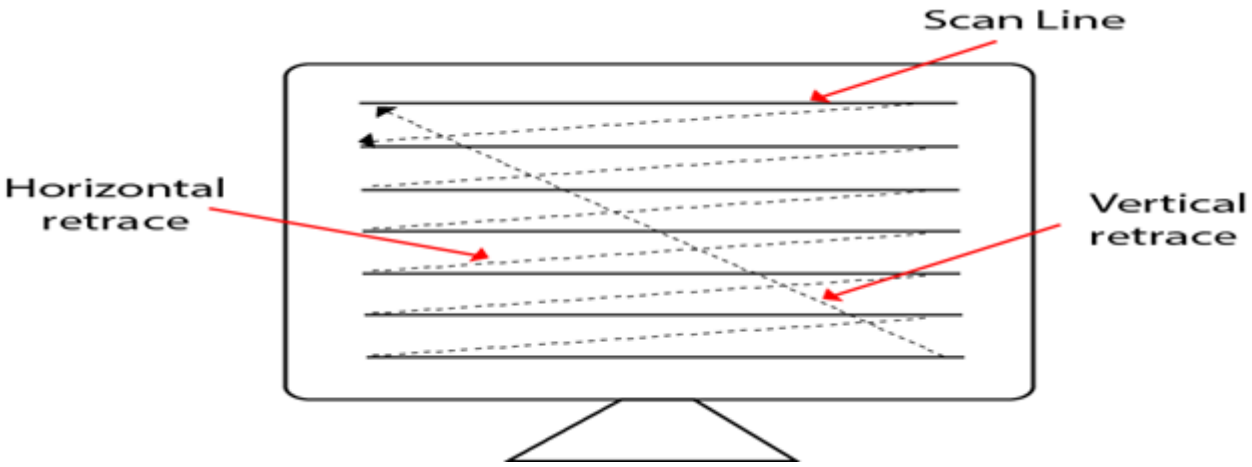
Disadvantages:

1. Random-Scan monitors cannot display realistic shades scenes.

Raster Scan Display:

A Raster Scan Display is based on intensity control of pixels in the form of a rectangular box called Raster on the screen. Information of on and off pixels is stored in refresh buffer or Frame buffer. Televisions in our house are based on Raster Scan Method. The raster scan system can store information of each pixel position, so it is suitable for realistic display of objects. Raster Scan provides a refresh rate of 60 to 80 frames per second.

Frame Buffer is also known as Raster or bit map. In Frame Buffer the positions are called picture elements or pixels. Beam refreshing is of two types. First is horizontal retracing and second is vertical retracing. When the beam starts from the top left corner and reaches the bottom right scale, it will again return to the top left side called at vertical retrace. Then it will again more horizontally from top to bottom call as horizontal retracing shown in fig:



Types of Scanning or travelling of beam in Raster Scan

1. Interlaced Scanning
2. Non-Interlaced Scanning

In Interlaced scanning, each horizontal line of the screen is traced from top to bottom. Due to which fading of display of object may occur. This problem can be solved by Non-Interlaced scanning. In this first of all odd numbered lines are traced or visited by an electron beam, then in the next circle, even number of lines are located.

For non-interlaced display refresh rate of 30 frames per second used. But it gives flickers. For interlaced display refresh rate of 60 frames per second is used.

Advantages:

1. Realistic image
2. Million Different colors to be generated
3. Shadow Scenes are possible.

Disadvantages:

1. Low Resolution
2. Expensive

Differentiate between Random and Raster Scan Display:

Random Scan	**Raster Scan**
1. It has high Resolution	1. Its resolution is low.
2. It is more expensive	2. It is less expensive
3. Any modification if needed is easy	3.Modification is tough
4. Solid pattern is tough to fill	4.Solid pattern is easy to fill
5. Refresh rate depends or resolution	5. Refresh rate does not depend on the picture.
6. Only screen with view on an area is displayed.	6. Whole screen is scanned.
7. Beam Penetration technology come under it.	7. Shadow mark technology came under this.
8. It does not use interlacing method.	8. It uses interlacing
9. It is restricted to line drawing applications	9. It is suitable for realistic display.

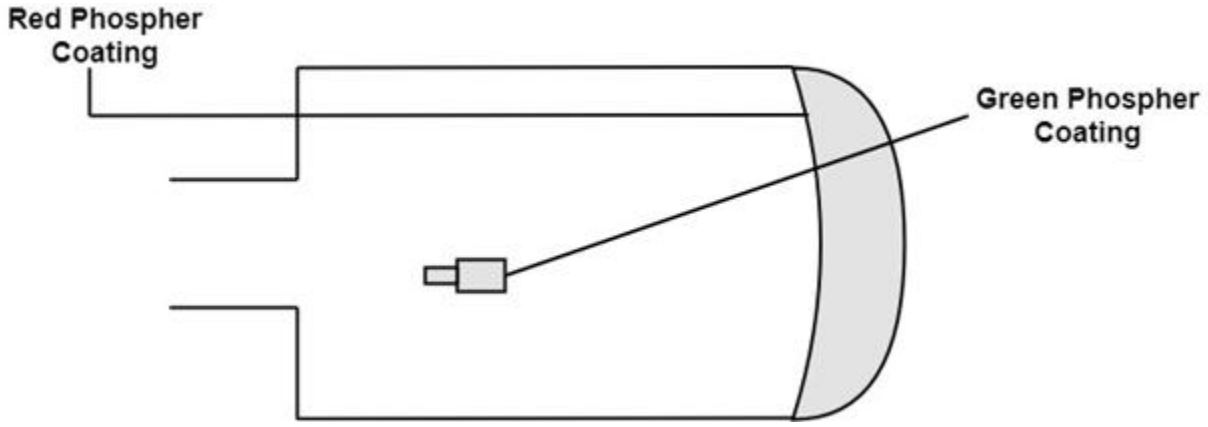
Color CRT Monitors:

The CRT Monitor display by using a combination of phosphors. The phosphors are different colors. There are two popular approaches for producing color displays with a CRT are:

1. Beam Penetration Method
2. Shadow-Mask Method

1. Beam Penetration Method:

The Beam-Penetration method has been used with random-scan monitors. In this method, the CRT screen is coated with two layers of phosphor, red and green and the displayed color depends on how far the electron beam penetrates the phosphor layers. This method produces four colors only, red, green, orange and yellow. A beam of slow electrons excites the outer red layer only; hence screen shows red color only. A beam of high-speed electrons excites the inner green layer. Thus screen shows a green color.



Advantages:

1. Inexpensive

Disadvantages:

1. Only four colors are possible
2. Quality of pictures is not as good as with another method.

2. Shadow-Mask Method:

- Shadow Mask Method is commonly used in Raster-Scan System because they produce a much wider range of colors than the beam-penetration method.
- It is used in the majority of color TV sets and monitors.

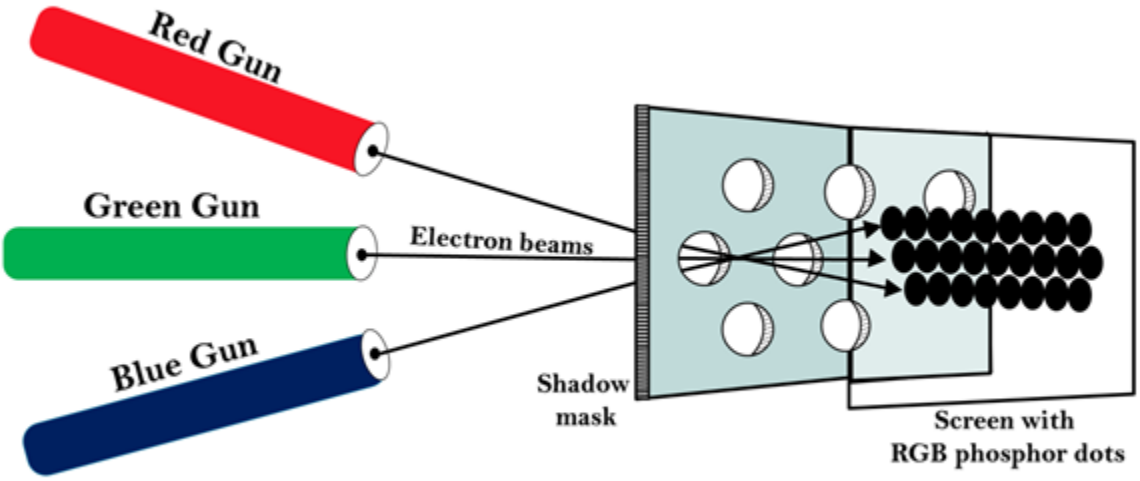
Construction: A shadow mask CRT has 3 phosphor color dots at each pixel position.

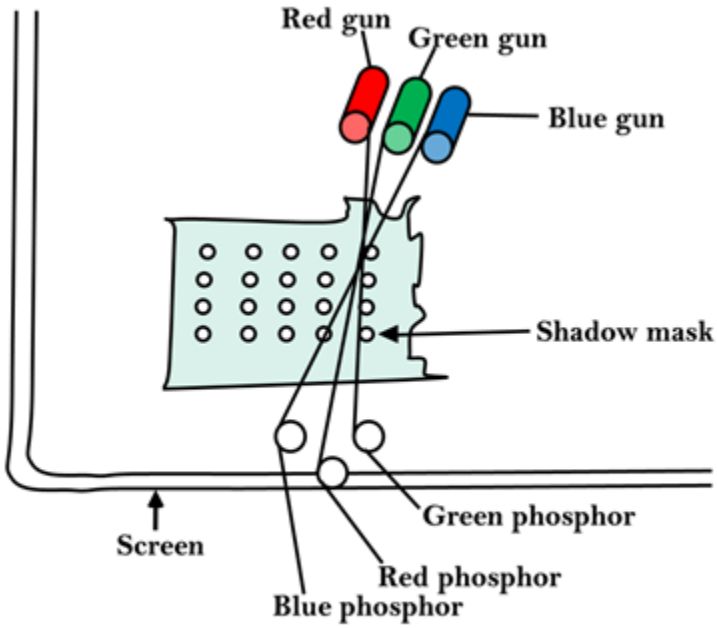
- One phosphor dot emits: red light
- Another emits: green light
- Third emits: blue light

This type of CRT has 3 electron guns, one for each color dot and a shadow mask grid just behind the phosphor coated screen.

Shadow mask grid is pierced with small round holes in a triangular pattern.

Figure shows the delta-delta shadow mask method commonly used in color CRT system.





The Shadow mask CRT

Working: Triad arrangement of red, green, and blue guns.

The deflection system of the CRT operates on all 3 electron beams simultaneously; the 3 electron beams are deflected and focused as a group onto the shadow mask, which contains a sequence of holes aligned with the phosphor- dot patterns.

When the three beams pass through a hole in the shadow mask, they activate a dotted triangle, which occurs as a small color spot on the screen.

The phosphor dots in the triangles are organized so that each electron beam can activate only its corresponding color dot when it passes through the shadow mask.

Inline arrangement: Another configuration for the 3 electron guns is an Inline arrangement in which the 3

electron guns and the corresponding red-green-blue color dots on the screen, are aligned along one scan line rather of in a triangular pattern.

This inline arrangement of electron guns in easier to keep in alignment and is commonly used in high-resolution color CRT's.

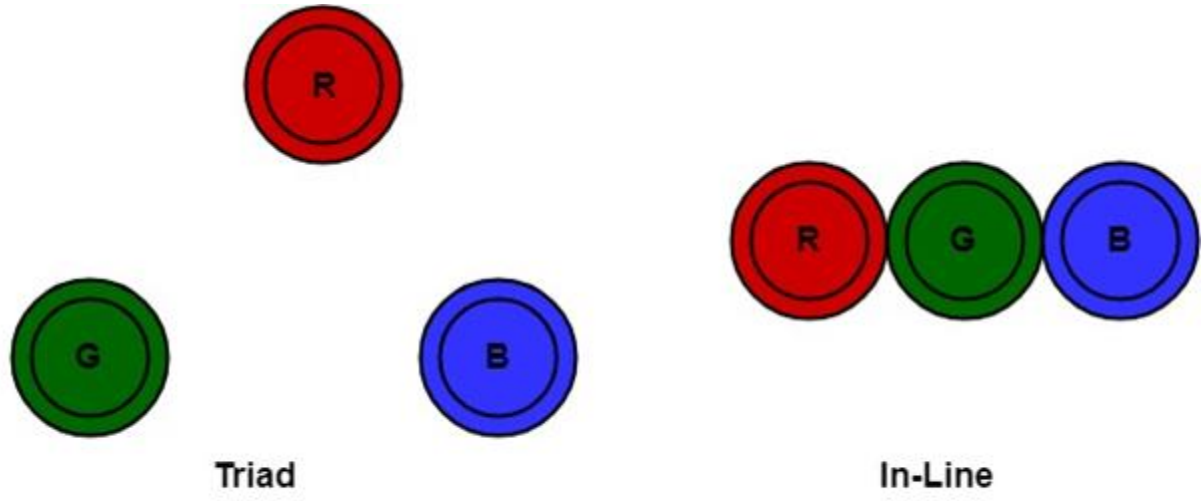


Fig: Triad-and -in-line arrangements of red, green and blue electron guns of CRT for color monitors.

Advantage:

1. Realistic image
2. Million different colors to be generated
3. Shadow scenes are possible

Disadvantage:

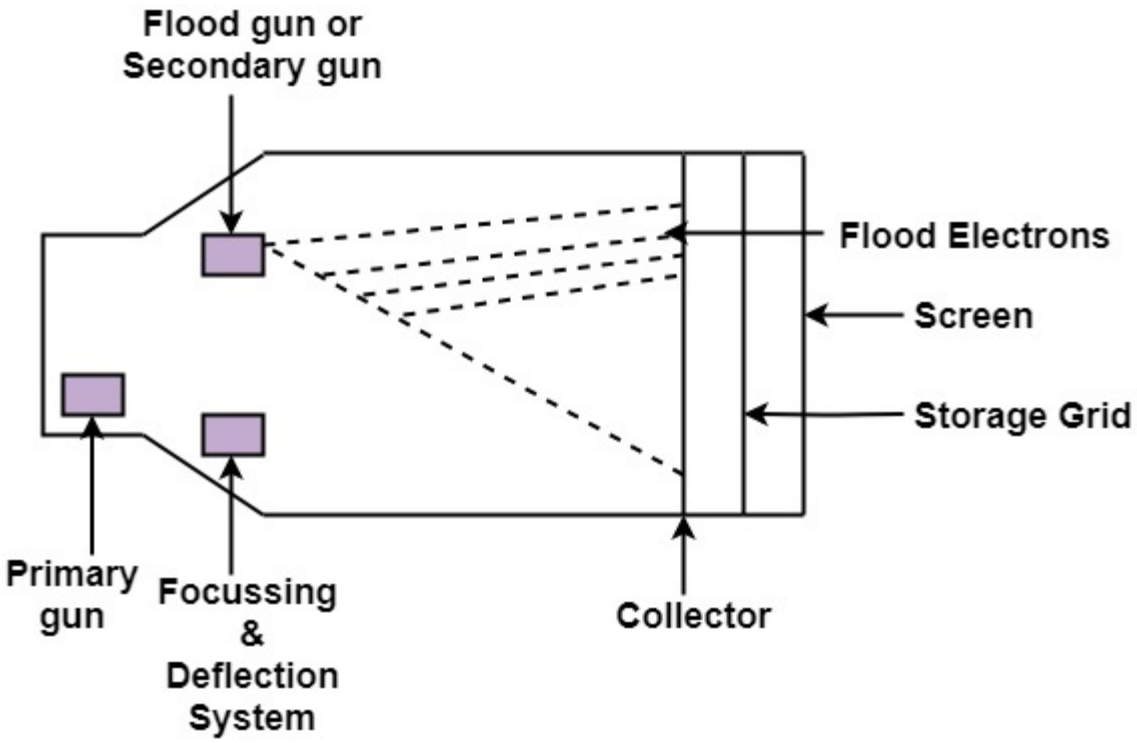
1. Relatively expensive compared with the monochrome CRT.
2. Relatively poor resolution
3. Convergence Problem

Direct View Storage Tubes:

DVST terminals also use the random scan approach to generate the image on the CRT screen. The term "storage tube" refers to the ability of the screen to retain the image which has been projected against it, thus avoiding the need to rewrite the image constantly.

Function of guns: Two guns are used in DVST

1. **Primary guns:** It is used to store the picture pattern.
2. **Flood gun or Secondary gun:** It is used to maintain picture display.



Direct View Storage Tube

Advantage:

1. No refreshing is needed.
2. High Resolution
3. Cost is very less

Disadvantage:

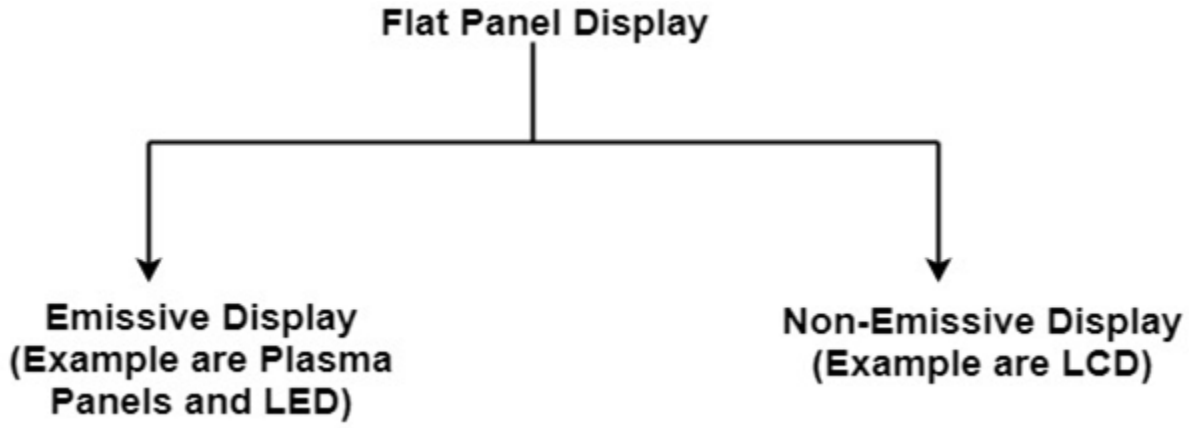
1. It is not possible to erase the selected part of a picture.
2. It is not suitable for dynamic graphics applications.

3. If a part of picture is to modify, then time is consumed.

Flat Panel Display:

The Flat-Panel display refers to a class of video devices that have reduced volume, weight and power requirement compare to CRT.

Example: Small T.V. monitor, calculator, pocket video games, laptop computers, an advertisement board in elevator.



1. Emissive Display: The emissive displays are devices that convert electrical energy into light. Examples are Plasma Panel, thin film electroluminescent display and LED (Light Emitting Diodes).

2. Non-Emissive Display: The Non-Emissive displays use optical effects to convert sunlight or light from some other source into graphics patterns. Examples are LCD (Liquid Crystal Device).

Plasma Panel Display:

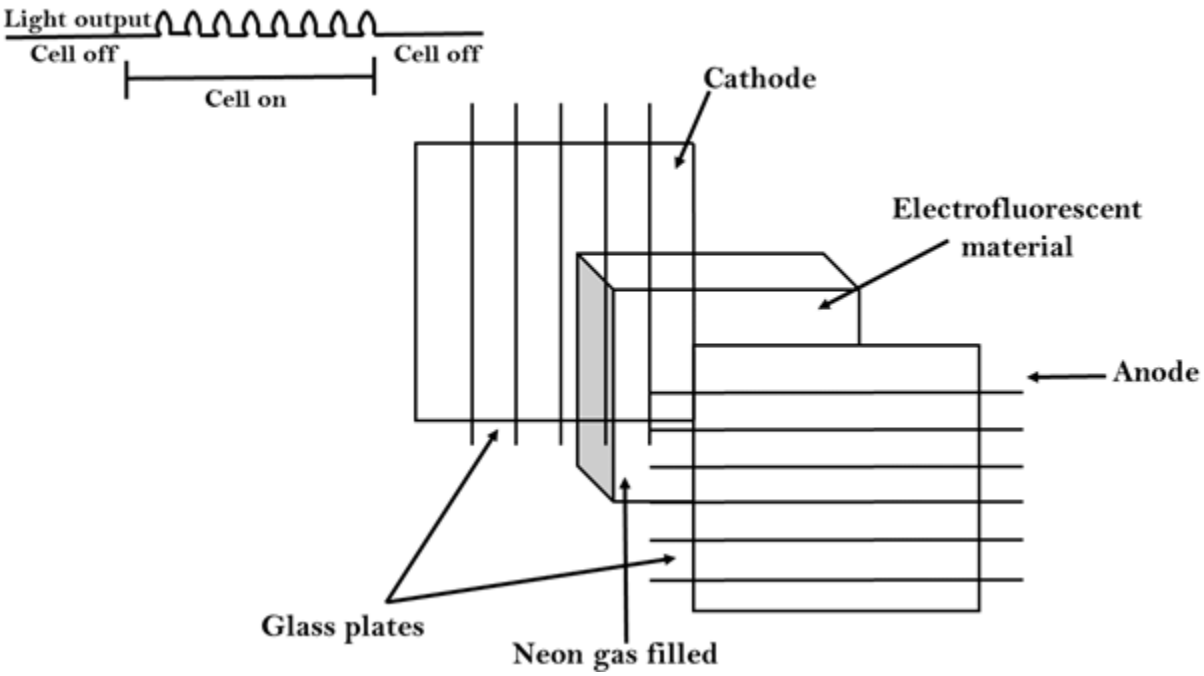
Plasma-Panels are also called as Gas-Discharge Display. It consists of an array of small lights. Lights are fluorescent in nature. The essential components of the plasma-panel display are:

1. **Cathode:** It consists of fine wires. It delivers negative voltage to gas cells. The voltage is released along with the negative axis.
2. **Anode:** It also consists of line wires. It delivers positive voltage. The voltage is supplied along positive axis.
3. **Fluorescent cells:** It consists of small pockets of gas liquids when the voltage is applied to this liquid (neon gas) it emits light.
4. **Glass Plates:** These plates act as capacitors. The voltage will be applied, the cell will glow continuously.

The gas will slow when there is a significant voltage difference between horizontal and vertical wires. The voltage level is kept between 90 volts to 120 volts. Plasma level does not require refreshing. Erasing is done by reducing the voltage to 90 volts.

Each cell of plasma has two states, so cell is said to be stable. Displayable point in plasma panel is made by the crossing of the horizontal and vertical grid. The resolution of the plasma panel can be up to 512 * 512 pixels.

Figure shows the state of cell in plasma panel display:



Advantage:

1. High Resolution
2. Large screen size is also possible.
3. Less Volume
4. Less weight
5. Flicker Free Display

Disadvantage:

1. Poor Resolution
2. Wiring requirement anode and the cathode is complex.
3. Its addressing is also complex.

LED (Light Emitting Diode):

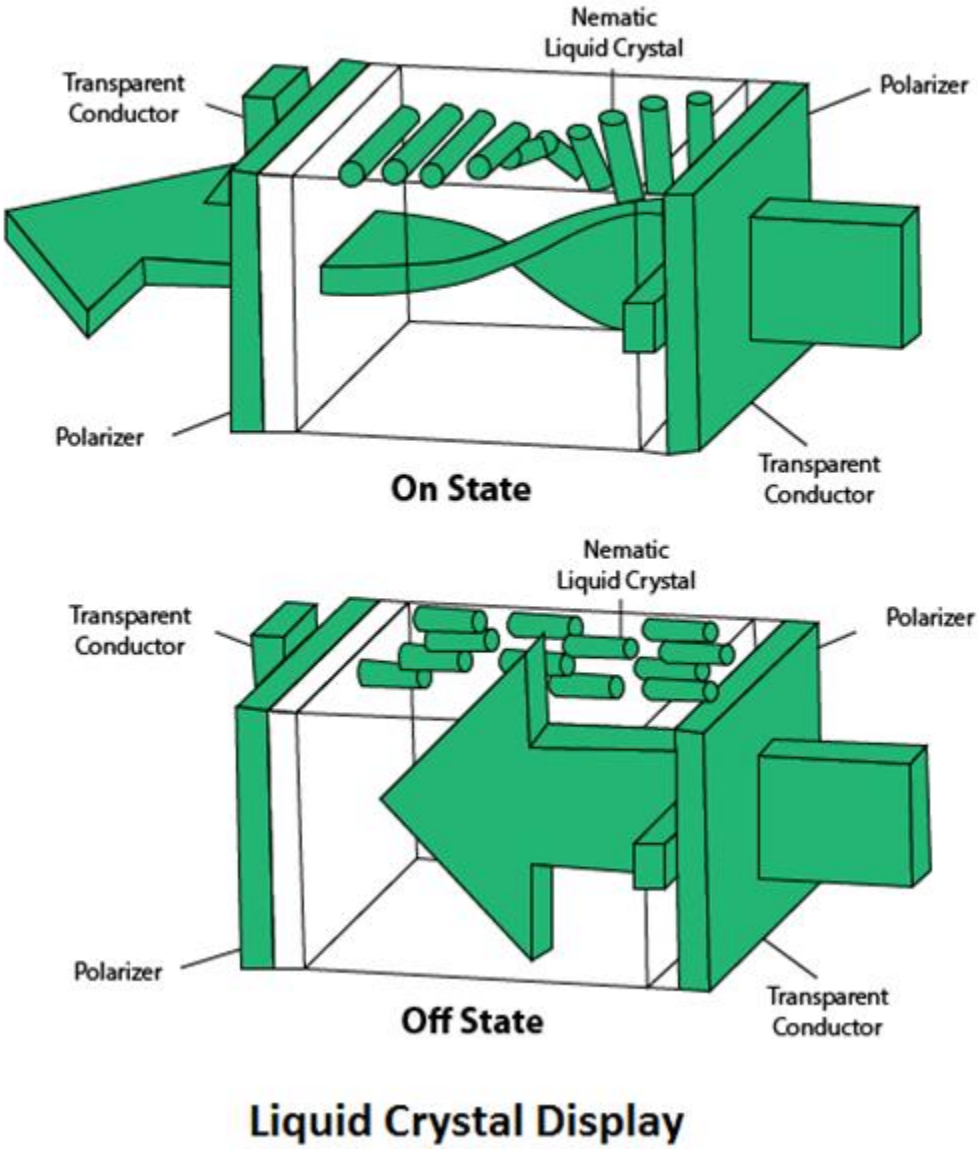
In an LED, a matrix of diodes is organized to form the pixel positions in the display and picture definition is stored in a refresh buffer. Data is read from the refresh buffer and converted to voltage levels that are applied to the diodes to produce the light pattern in the display.

LCD (Liquid Crystal Display):

Liquid Crystal Displays are the devices that produce a picture by passing polarized light from the surroundings or from an internal light source through a liquid-crystal material that transmits the light.

LCD uses the liquid-crystal material between two glass plates; each plate is the right angle to each other between plates liquid is filled. One glass plate consists of rows of conductors arranged in vertical direction. Another glass plate is consisting of a row of conductors arranged in horizontal direction. The pixel position is determined by the intersection of the vertical & horizontal conductor. This position is an active part of the screen.

Liquid crystal display is temperature dependent. It is between zero to seventy degree Celsius. It is flat and requires very little power to operate.



Liquid Crystal Display

Advantage:

1. Low power consumption.
2. Small Size
3. Low Cost

Disadvantage:

1. LCDs are temperature-dependent (0-70°C)

2. LCDs do not emit light; as a result, the image has very little contrast.
3. LCDs have no color capability.
4. The resolution is not as good as that of a CRT.

Look-Up Table:

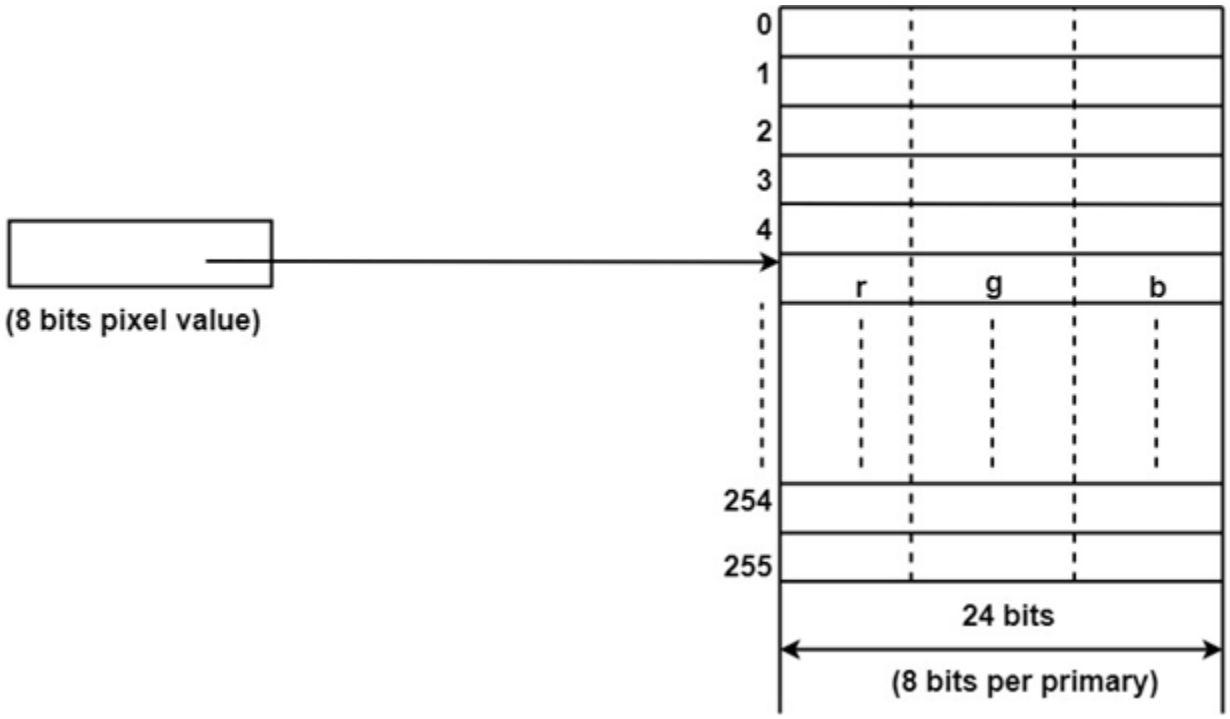
Image representation is essentially the description of pixel colors. There are three primary colors: R (red), G (green) and B (blue). Each primary color can take on intensity levels produces a variety of colors. Using direct coding, we may allocate 3 bits for each pixel, with one bit for each primary color. The 3-bit representation allows each primary to vary independently between two intensity levels: 0 (off) or 1 (on). Hence each pixel can take on one of the eight colors.

Bit 1:r	**Bit 2:g**	**Bit 3:b**	**Color name**
0	0	0	Black
0	0	1	Blue
0	1	0	Green
0	1	1	Cyan
1	0	0	Red
1	0	1	Magenta
1	1	0	Yellow
1	1	1	White

A widely accepted industry standard uses 3 bytes, or 24 bytes, per pixel, with one byte for each primary color. The way, we allow each primary color to have 256 different intensity levels. Thus a pixel can take on a color from 256 x 256 x 256 or 16.7 million possible choices. The 24-bit format is commonly referred to as the actual color representation.

Lookup Table approach reduces the storage requirement. In this approach pixel values do not code colors directly. Alternatively, they are addresses or indices into a table of color values. The color

of a particular pixel is determined by the color value in the table entry that the value of the pixel references. Figure shows a look-up table with 256 entries. The entries have addresses 0 through 255. Each entry contains a 24-bit RGB color value. Pixel values are now 1-byte. The color of a pixel whose value is i, where 0 <i<255, is persistence by the color value in the table entry whose address is i. It reduces the storage requirement of a 1000 x 1000 image to one million bytes plus 768 bytes for the color values in the look-up table.



Scan Conversion Definition

It is a process of representing graphics objects a collection of pixels. The graphics objects are continuous. The pixels used are discrete. Each pixel can have either on or off state.

The circuitry of the video display device of the computer is capable of converting binary values (0, 1) into a pixel on and pixel off information. 0 is represented by pixel off. 1 is represented using pixel on. Using this ability graphics computer represent picture having discrete dots.

Any model of graphics can be reproduced with a dense matrix of dots or points. Most human beings think graphics objects as points, lines, circles, ellipses. For generating graphical object, many algorithms have been developed.

Advantage of developing algorithms for scan conversion

1. Algorithms can generate graphics objects at a faster rate.
2. Using algorithms memory can be used efficiently.
3. Algorithms can develop a higher level of graphical objects.

Examples of objects which can be scan converted

1. Point
2. Line
3. Sector
4. Arc
5. Ellipse
6. Rectangle
7. Polygon
8. Characters
9. Filled Regions

The process of converting is also called as rasterization. The algorithms implementation varies from one computer system to another computer system. Some algorithms are implemented using the software. Some are performed using hardware or firmware. Some are performed using various combinations of hardware, firmware, and software.

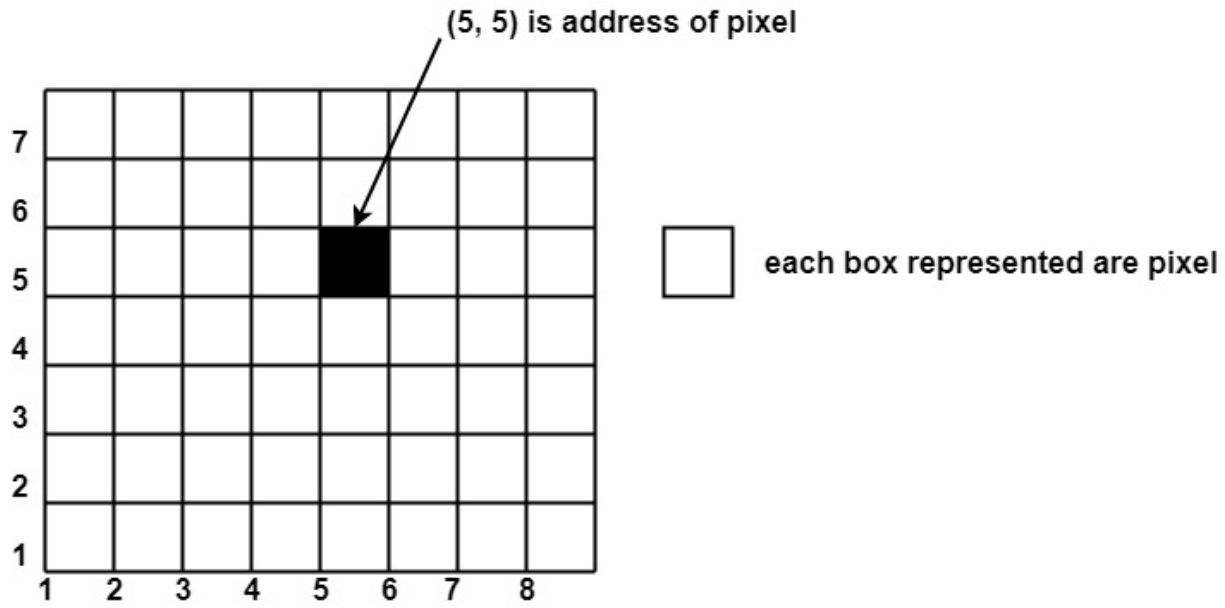
Pixel or Pel:

The term pixel is a short form of the picture element. It is also called a point or dot. It is the smallest picture unit accepted by display devices. A picture is constructed from hundreds of such pixels.

Pixels are generated using commands. Lines, circle, arcs, characters; curves are drawn with closely spaced pixels. To display the digit or letter matrix of pixels is used.

The closer the dots or pixels are, the better will be the quality of picture. Closer the dots are, crisper will be the picture. Picture will not appear jagged and unclear if pixels are closely spaced. So the quality of the picture is directly proportional to the density of pixels on the screen.

Pixels are also defined as the smallest addressable unit or element of the screen. Each pixel can be assigned an address as shown in fig:



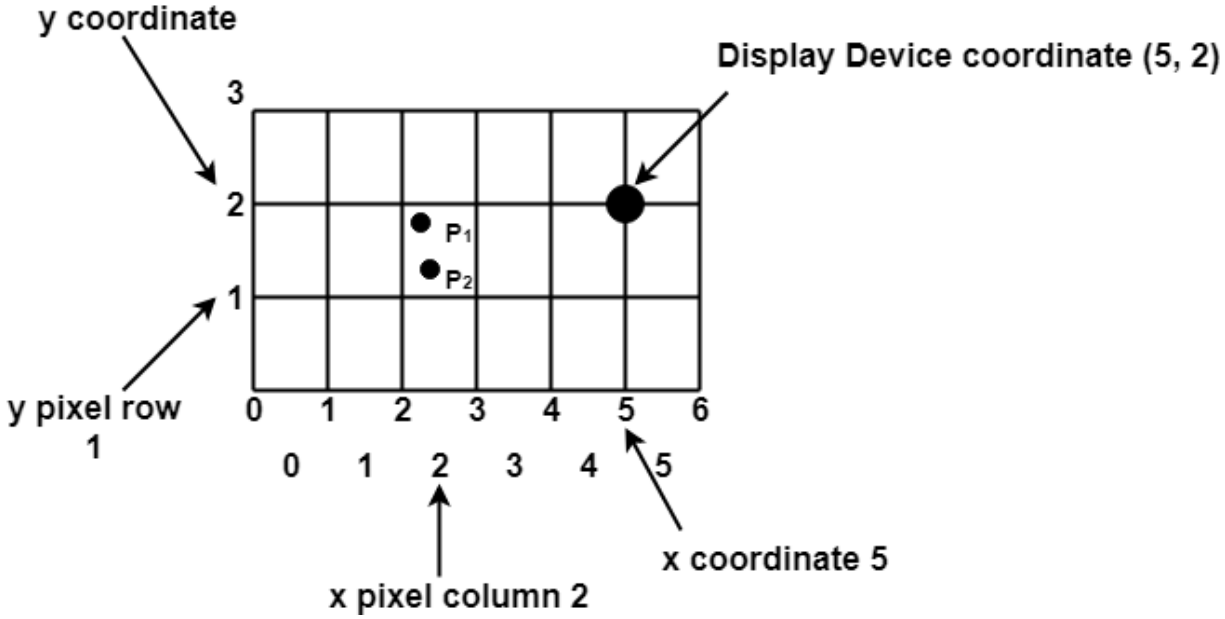
Different graphics objects can be generated by setting the different intensity of pixels and different colors of pixels. Each pixel has some co-ordinate value. The coordinate is represented using row and column.

P (5, 5) used to represent a pixel in the 5th row and the 5th column. Each pixel has some intensity value which is represented in memory of computer called a **frame buffer**. Frame Buffer is also called a refresh buffer. This memory is a storage area for storing pixels values using which pictures are displayed. It is also called as digital memory. Inside the buffer, image is stored as a pattern of binary digits either 0 or 1. So there is an array of 0 or 1 used to represent the picture. In black and white monitors, black pixels are represented using 1's and white pixels are represented using 0's. In case of systems having one bit per pixel frame buffer is called a bitmap. In systems with multiple bits per pixel it is called a pixmap.

Scan Converting a Point

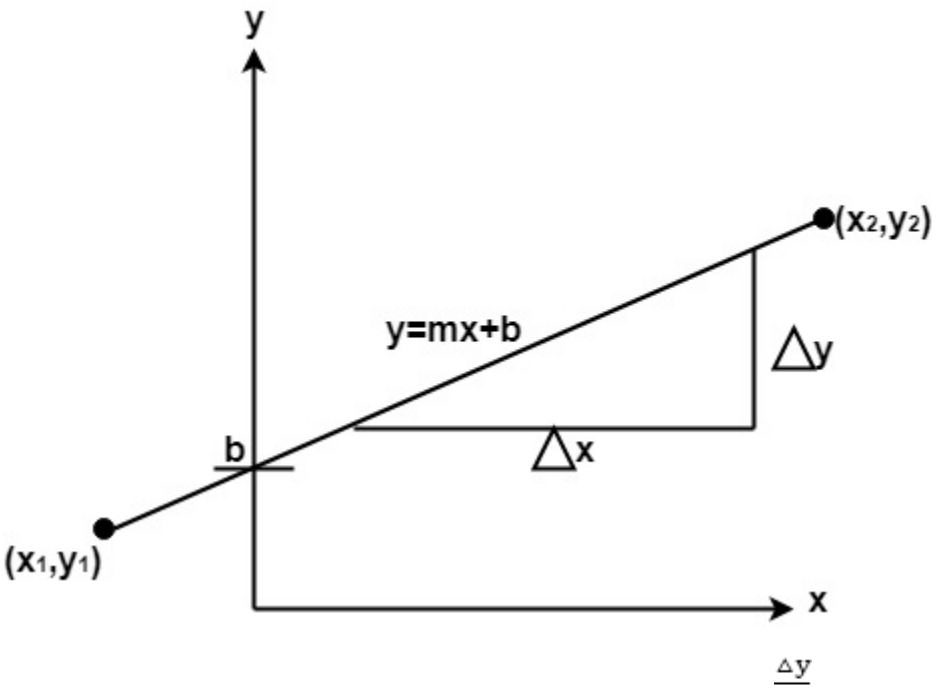
Each pixel on the graphics display does not represent a mathematical point. Instead, it means a region which theoretically can contain an infinite number of points. Scan-Converting a point involves illuminating the pixel that contains the point.

Example: Display coordinates points $P_1\ (2\frac{1}{4}, 1\frac{3}{4})$& $P_2(2\frac{2}{3}, 1\frac{1}{4})$as shown in fig would both be represented by pixel (2, 1). In general, a point p (x, y) is represented by the integer part of x & the integer part of y that is pixels [(INT (x), INT (y).



Scan Converting a Straight Line:

A straight line may be defined by two endpoints & an equation. In fig the two endpoints are described by (x_1,y_1) and (x_2,y_2). The equation of the line is used to determine the x, y coordinates of all the points that lie between these two endpoints.



Using the equation of a straight line, $y = mx + b$ where $m = \frac{\Delta y}{\Delta x}$ & b = the y interrupt, we can find values of y by incrementing x from $x = x_1$, to $x = x_2$. By scan-converting these calculated x, y values, we represent the line as a sequence of pixels.

Properties of Good Line Drawing Algorithm:

1. Line should appear Straight: We must appropriate the line by choosing addressable points close to it. If we choose well, the line will appear straight, if not, we shall produce crossed lines.

Fig: O/P from a poor line generating algorithm

The lines must be generated parallel or at 45° to the x and y-axes. Other lines cause a problem: a line segment through it starts and finishes at addressable points, may happen to pass through no another addressable points in between.

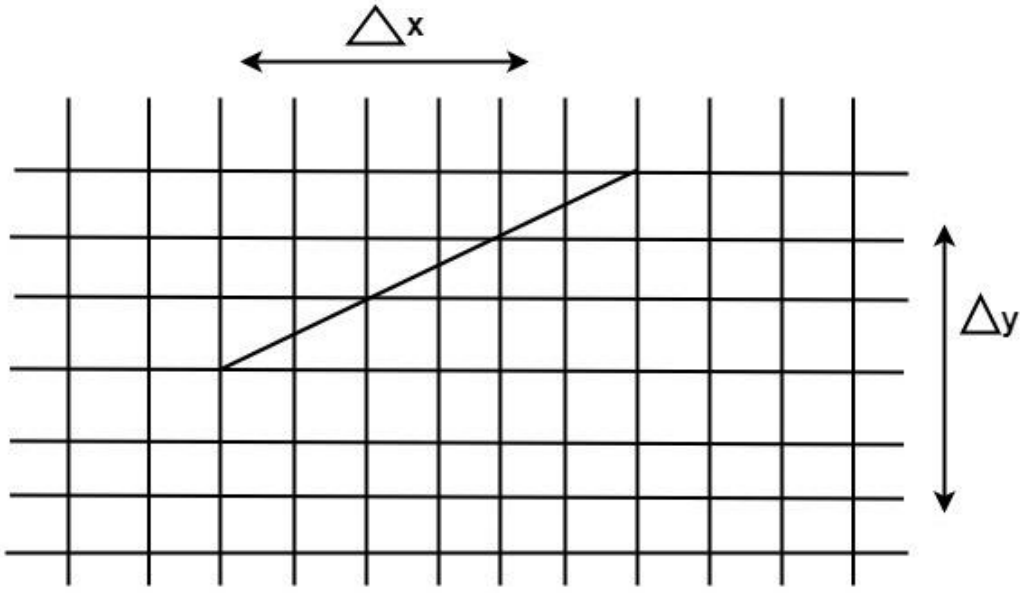


Fig: A straight line segment connecting 2 grid intersection may fail to pass through any other grid intersections.

2. Lines should terminate accurately: Unless lines are plotted accurately, they may terminate at the wrong place.

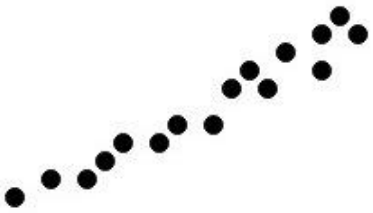


Fig: Uneven line density caused by bunching of dots.

3. Lines should have constant density: Line density is proportional to the no. of dots displayed divided by the length of the line.

To maintain constant density, dots should be equally spaced.

4. Line density should be independent of line length and angle: This can be done by computing an approximating line-length estimate and to use a line-generation algorithm that keeps line density constant to within the accuracy of this estimate.

5. Line should be drawn rapidly: This computation should be performed by special-purpose hardware.

Algorithm for line Drawing:

1. Direct use of line equation
2. DDA (Digital Differential Analyzer)
3. Bresenham's Algorithm

Direct use of line equation:

It is the simplest form of conversion. First of all scan P_1 and P_2 points. P_1 has co-ordinates (x_1',y_1') and (x_2' y_2').

Then m = (y_2',y_1')/(x_2',x_1') and b = $y_1^1 + mx_1^1$

If value of |m|≤1 for each integer value of x. But do not consider x_1^1 and x_2^2

If value of |m|>1 for each integer value of y. But do not consider y_1^1 and y_2^2

Example: A line with starting point as (0, 0) and ending point (6, 18) is given. Calculate value of intermediate points and slope of line.

Solution: P_1 (0,0) P_7 (6,18)

x_1=0

y_1=0

x_2=6

y_2=18

$$M = \frac{\Delta y}{\Delta x} = \frac{y_2 - y_1}{x_2 - x_1} = \frac{18-0}{6-0} = \frac{18}{6} = 3$$

We know equation of line is

y =m x + b

y = 3x + b.....equation (1)

put value of x from initial point in equation (1), i.e., (0, 0) x =0, y=0

0 = 3 x 0 + b

0 = b ⟹ b=0

put b = 0 in equation (1)

$$y = 3x + 0$$

$$y = 3x$$

Now calculate intermediate points

Let x = 1 $\Rightarrow$ y = 3 x 1 $\Rightarrow$ y = 3

Let x = 2 $\Rightarrow$ y = 3 x 2 $\Rightarrow$ y = 6

Let x = 3 $\Rightarrow$ y = 3 x 3 $\Rightarrow$ y = 9

Let x = 4 $\Rightarrow$ y = 3 x 4 $\Rightarrow$ y = 12

Let x = 5 $\Rightarrow$ y = 3 x 5 $\Rightarrow$ y = 15

Let x = 6 $\Rightarrow$ y = 3 x 6 $\Rightarrow$ y = 18

So points are

P_1 (0,0)

P_2 (1,3)

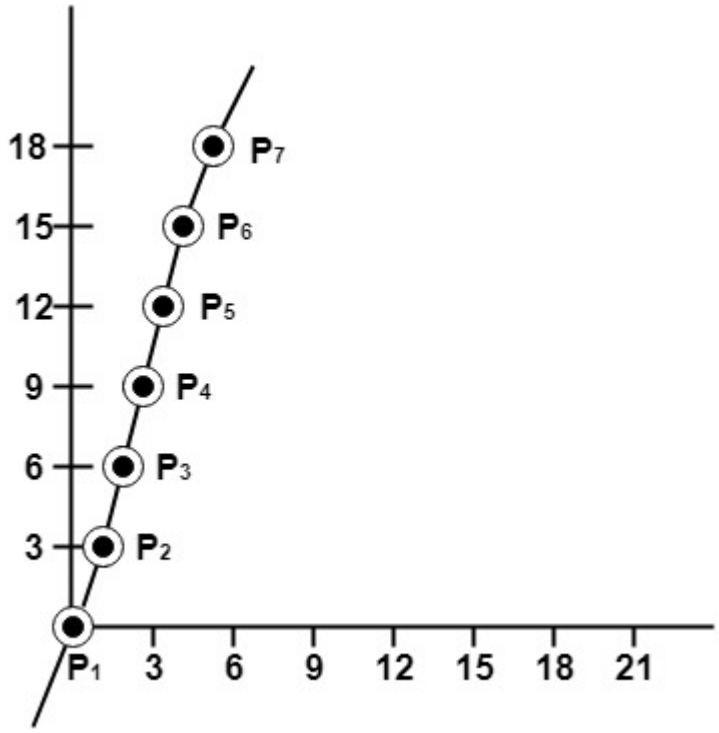
P_3 (2,6)

P_4 (3,9)

P_5 (4,12)

P_6 (5,15)

P_7 (6,18)



Algorithm for drawing line using equation:

Step1: Start Algorithm

Step2: Declare variables x_1,x_2,y_1,y_2,dx,dy,m,b,

Step3: Enter values of x_1,x_2,y_1,y_2.
The (x_1,y_1) are co-ordinates of a starting point of the line.
The (x_2,y_2) are co-ordinates of a ending point of the line.

Step4: Calculate $dx = x_2 - x_1$

Step5: Calculate $dy = y_2 - y_1$

Step6: Calculate $m = \frac{dy}{dx}$

Step7: Calculate $b = y_1 - m * x_1$

Step8: Set (x, y) equal to starting point, i.e., lowest point and x_{end}equal to largest value of x.

If dx < 0
then x = x_2
y = y_2
x_{end}= x_1
If dx > 0
then x = x_1
y = y_1
x_{end}= x_2

Step9: Check whether the complete line has been drawn if $x=x_{end}$, stop

Step10: Plot a point at current (x, y) coordinates

Step11: Increment value of x, i.e., x = x+1

Step12: Compute next value of y from equation y = mx + b

Step13: Go to Step9.

Program to draw a line using LineSlope Method

```
1. #include <graphics.h>
2. #include <stdlib.h>
3. #include <math.h>
4. #include <stdio.h>
5. #include <conio.h>
6. #include <iostream.h>
7.
```

```
8.  class bresen
9.  {
10. float x, y, x1, y1, x2, y2, dx, dy, m, c, xend;
11. public:
12. void get ();
13. void cal ();
14. };
15. void main ()
16. {
17. bresen b;
18. b.get ();
19. b.cal ();
20. getch ();
21. }
22. Void bresen :: get ()
23. {
24. print ("Enter start & end points");
25. print ("enter x1, y1, x2, y2");
26. scanf ("%f%f%f%f",sx1, sx2, sx3, sx4)
27. }
28. void bresen ::cal ()
29. {
30. /* request auto detection */
31. int gdriver = DETECT,gmode, errorcode;
32. /* initialize graphics and local variables */
33. initgraph (&gdriver, &gmode, " ");
34. /* read result of initialization */
```

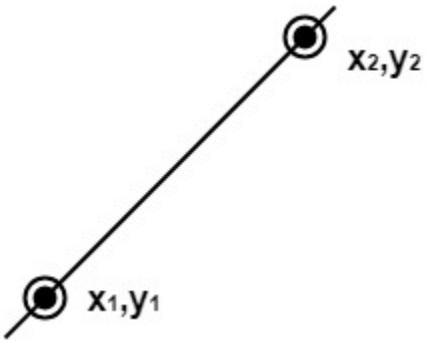
```
35. errorcode = graphresult ();
36. if (errorcode ! = grOK)    /*an error occurred */
37. {
38. printf("Graphics error: %s \n", grapherrormsg (errorcode);
39. printf ("Press any key to halt:");
40. getch ();
41. exit (1); /* terminate with an error code */
42. }
43. dx = x2-x1;
44. dy=y2-2y1;
45. m = dy/dx;
46. c = y1 - (m * x1);
47. if (dx<0)
48. {
49. x=x2;
50. y=y2;
51. xend=x1;
52. }
53. else
54. {
55. x=x1;
56. y=y1;
57. xend=x2;
58. }
59. while (x<=xend)
60. {
61. putpixel (x, y, RED);
```

```
62. y++;
63. y=(x*x) +c;
64. }
65. }
```

OUTPUT:

Enter Starting and End Points

Enter (X1, Y1, X2, Y2) 200 100 300 200



DDA Algorithm

DDA stands for Digital Differential Analyzer. It is an incremental method of scan conversion of line. In this method calculation is performed at each step but by using results of previous steps.

Suppose at step i, the pixels is (x_i,y_i)

The line of equation for step i $y_i=mx_{i+b}$.....equation 1

Next value will be $y_{i+1}=m_{xi+1}+b$.....equation 2

$m = \frac{\Delta y}{\Delta x}$

$y_{i+1}-y_i=\Delta y$.....equation 3

$y_{i+1}-x_i=\Delta x$.....equation 4

$y_{i+1}=y_i+\Delta y$

$\Delta y=m\Delta x$

$y_{i+1}=y_i+m\Delta x$

$\Delta x=\Delta y/m$

$x_{i+1}=x_i+\Delta x$

$x_{i+1}=x_i+\Delta y/m$

Case1: When |M|<1 then (assume that $x_1<x_2$) $x=x_1, y=y_1$ set $\Delta x=1$

$y_{i+1}=y_{1+m}$, $x=x+1$

Until x = x_2</x

Case2: When |M|<1 then (assume that $y_1<y_2$) $x=x_1, y=y_1$ set $\Delta y=1$

$x_{i+1}=\frac{1}{m}$, $y=y+1$

Until y → y_2</y

Advantage:

1. It is a faster method than method of using direct use of line equation.
2. This method does not use multiplication theorem.
3. It allows us to detect the change in the value of x and y ,so plotting of same point twice is not possible.
4. This method gives overflow indication when a point is repositioned.
5. It is an easy method because each step involves just two additions.

Disadvantage:

1. It involves floating point additions rounding off is done. Accumulations of round off error cause accumulation of error.
2. Rounding off operations and floating point operations consumes a lot of time.
3. It is more suitable for generating line using the software. But it is less suited for hardware implementation.

DDA Algorithm:

Step1: Start Algorithm

Step2: Declare x_1,y_1,x_2,y_2,dx,dy,x,y as integer variables.

Step3: Enter value of x_1,y_1,x_2,y_2.

Step4: Calculate dx = x_2-x_1

Step5: Calculate dy = y_2-y_1

Step6: If ABS (dx) > ABS (dy)
Then step = abs (dx)
Else

Step7: x_{inc}=dx/step
y_{inc}=dy/step
assign x = x_1
assign y = y_1

Step8: Set pixel (x, y)

Step9: x = x + x_{inc}
y = y + y_{inc}
Set pixels (Round (x), Round (y))

Step10: Repeat step 9 until x = x_2

Step11: End Algorithm

Example: If a line is drawn from (2, 3) to (6, 15) with use of DDA. How many points will needed to generate such line?

Solution: P_1 (2,3) P_{11} (6,15)

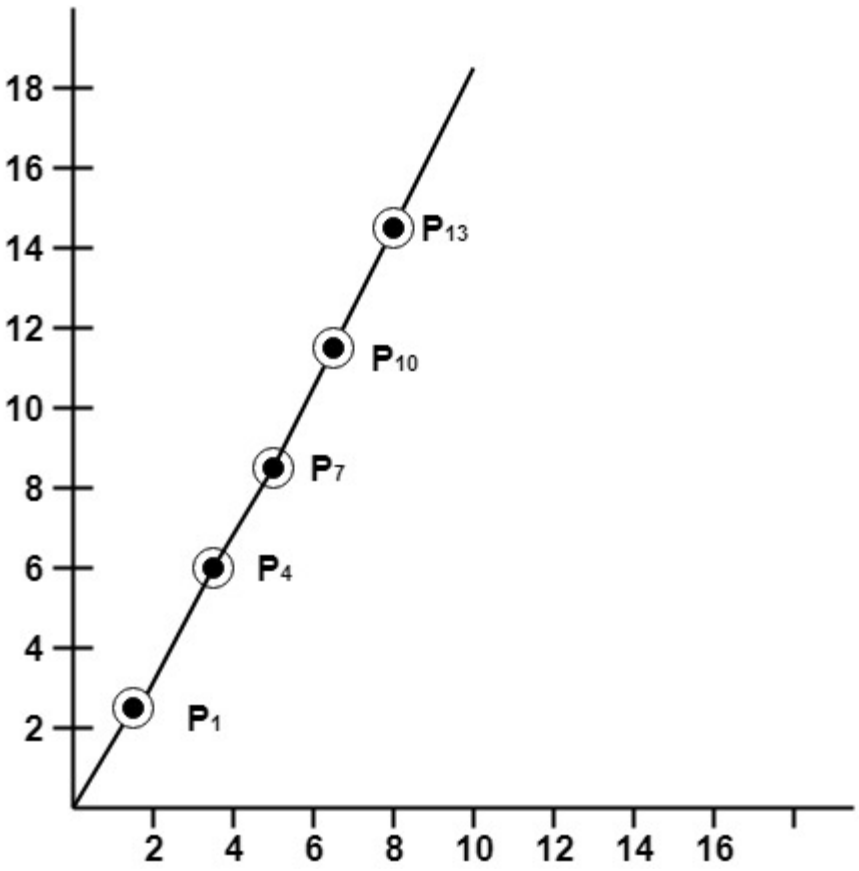
x_1=2
y_1=3
x_2= 6
y_2=15
dx = 6 - 2 = 4

dy = 15 - 3 = 12

$$m = \frac{dy}{dx} = \frac{12}{4}$$

For calculating next value of x takes $x = x + \frac{1}{m}$

$P_1(2, 3)$	point plotted
$P_2(2\frac{1}{3}, 4)$	point plotted
$P_3(2\frac{2}{3}, 5)$	point not plotted
$P_4(3, 6)$	point plotted
$P_5(3\frac{1}{3}, 7)$	point not plotted
$P_6(3\frac{2}{3}, 8)$	point not plotted
$P_7(4, 9)$	point plotted
$P_8(4\frac{1}{3}, 10)$	point not plotted
$P_9(4\frac{2}{3}, 11)$	point not plotted
$P_{10}(5, 12)$	point plotted
$P_{11}(5\frac{1}{3}, 13)$	point not plotted
$P_{12}(5\frac{2}{3}, 14)$	point not plotted
$P_{13}(6, 15)$	point plotted

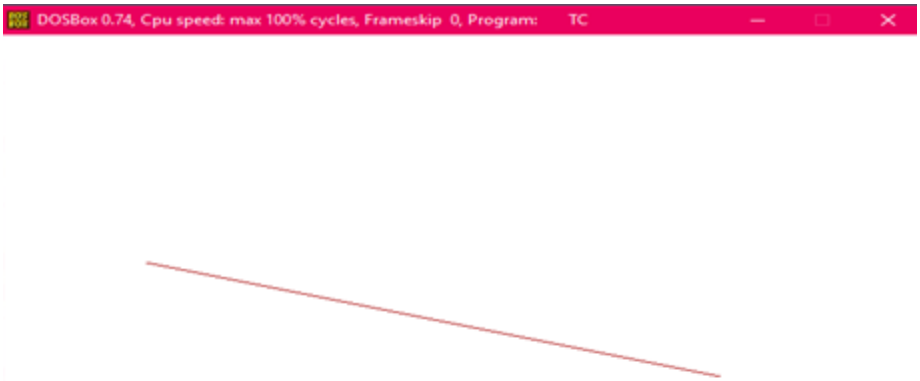


Program to implement DDA Line Drawing Algorithm:

```
1.  #include<graphics.h>
2.  #include<conio.h>
3.  #include<stdio.h>
4.  void main()
5.  {
6.  intgd = DETECT ,gm, i;
7.  float x, y,dx,dy,steps;
8.  int x0, x1, y0, y1;
9.  initgraph(&gd, &gm, "C:\\TC\\BGI");
10. setbkcolor(WHITE);
11. x0 = 100 , y0 = 200, x1 = 500, y1 = 300;
12. dx = (float)(x1 - x0);
```

```
13. dy = (float)(y1 - y0);
14. if(dx>=dy)
15. {
16. steps = dx;
17. }
18. else
19. {
20. steps = dy;
21. }
22. dx = dx/steps;
23. dy = dy/steps;
24. x = x0;
25. y = y0;
26. i = 1;
27. while(i<= steps)
28. {
29. putpixel(x, y, RED);
30. x += dx;
31. y += dy;
32. i=i+1;
33. }
34. getch();
35. closegraph();
36. }
```

Output:



Symmetrical DDA:

The Digital Differential Analyzer (DDA) generates lines from their differential equations. The equation of a straight line is

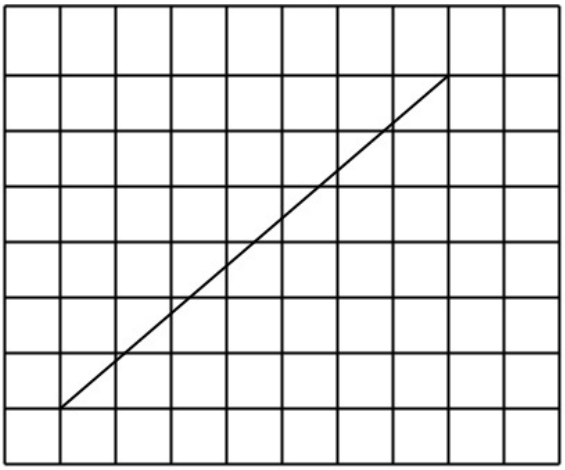
$$\frac{dy}{dx} = \frac{\Delta y}{\Delta x}$$

The DDA works on the principle that we simultaneously increment x and y by small steps proportional to the first derivatives of x and y. In this case of a straight line, the first derivatives are constant and are proportional to Δx and Δy . Therefore, we could generate a line by incrementing x and y by $\epsilon\,\Delta x$ and $\epsilon\,\Delta y$, where ϵ is some small quantity. There are two ways to generate points

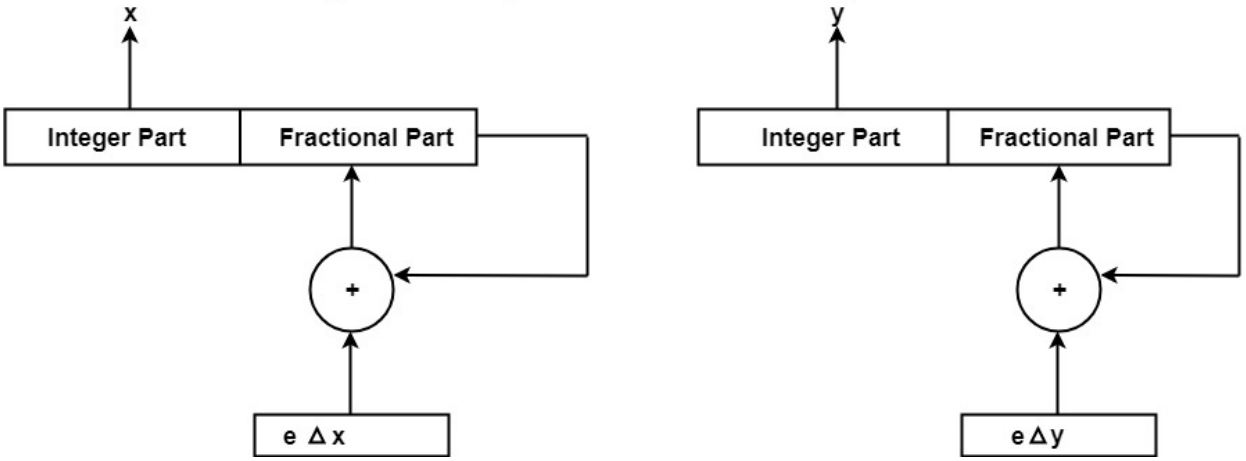
1. By rounding to the nearest integer after each incremental step, after rounding we display dots at the resultant x and y.

2. An alternative to rounding the use of arithmetic overflow: x and y are kept in registers that have two parts, integer and fractional. The incrementing values, which are both less than unity, are repeatedly added to the fractional parts and whenever the results overflows, the corresponding integer part is incremented. The integer parts of the x and y registers are used in plotting the line. In the case of the symmetrical DDA, we choose $\varepsilon=2^{-n}$,where $2^{n-1}\leq\max(|\Delta x|,|\Delta y|)<2^{\pi}$

A line drawn with the symmetrical DDA is shown in fig:



Organization of Symmetrical DDA shown in fig:



Example: If a line is drawn from (0, 0) to (10, 5) with a symmetrical DDA

1. How many iterations are performed?

2. How many different points will be generated?

Solutions: Given: P^1 (0,0) P^2 (10,5)

$x^1=0$

$y^1=0$

$x^2=10$

$y^2=5$

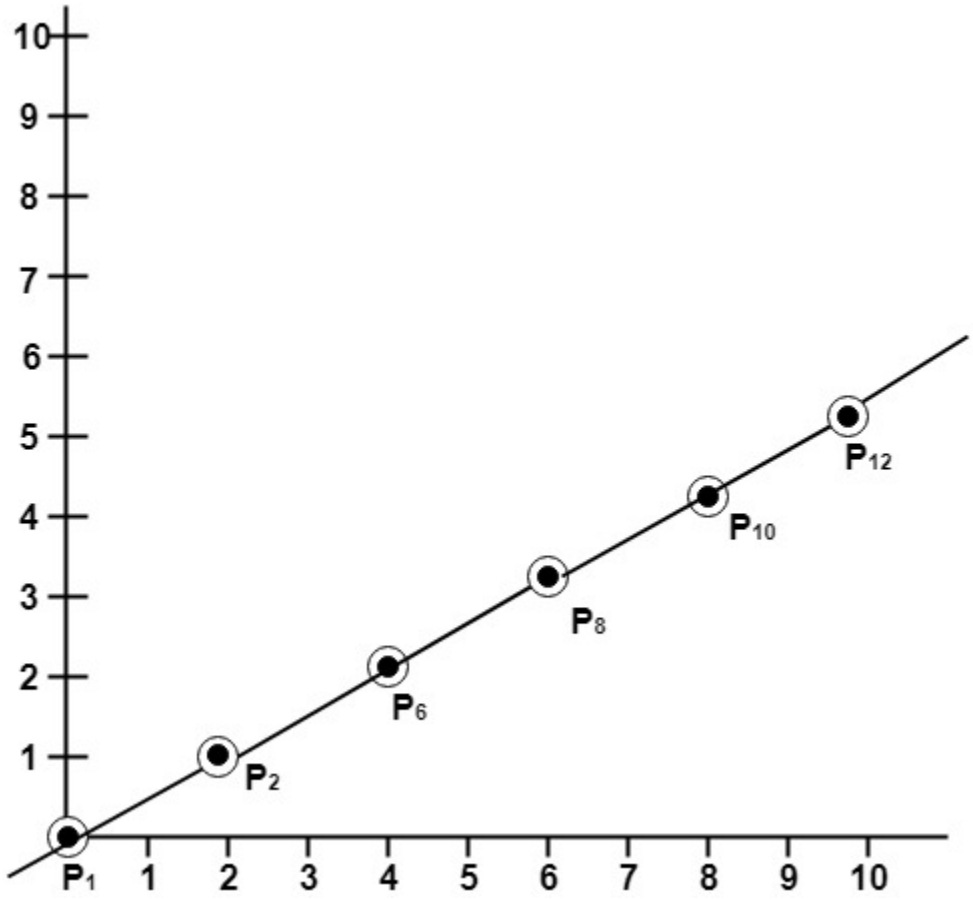
dx = 10 - 0 = 10

dy = 5 - 0 = 0

$$m = \frac{y_2-y_1}{x_2-x_1} = \frac{5-0}{10-0} = \frac{5}{10} = 0.5$$

P^1 (0,0) will be considered starting points
P^3 (1,0.5) point not plotted
P^4 (2, 1) point plotted
P^5 (3, 1.5) point not plotted
P^6 (4,2) point plotted
P^7 (5,2.5) point not plotted
P^8 (6,3) point plotted
P^9 (7,3.5) point not plotted
P^{10} (8, 4) point plotted
P^{11} (9,4.5) point not plotted
P^{12} (10,5) point plotted

Following Figure show line plotted using these points.



Bresenham's Line Algorithm

This algorithm is used for scan converting a line. It was developed by Bresenham. It is an efficient method because it involves only integer addition, subtractions, and multiplication operations. These operations can be performed very rapidly so lines can be generated quickly.

In this method, next pixel selected is that one who has the least distance from true line.

The method works as follows:

Assume a pixel $P_1'(x_1',y_1')$,then select subsequent pixels as we work our may to the night, one pixel position at a time in the horizontal direction toward $P_2'(x_2',y_2')$.

Once a pixel in choose at any step

The next pixel is

1. Either the one to its right (lower-bound for the line)
2. One top its right and up (upper-bound for the line)

The line is best approximated by those pixels that fall the least distance from the path between P_1',P_2'.

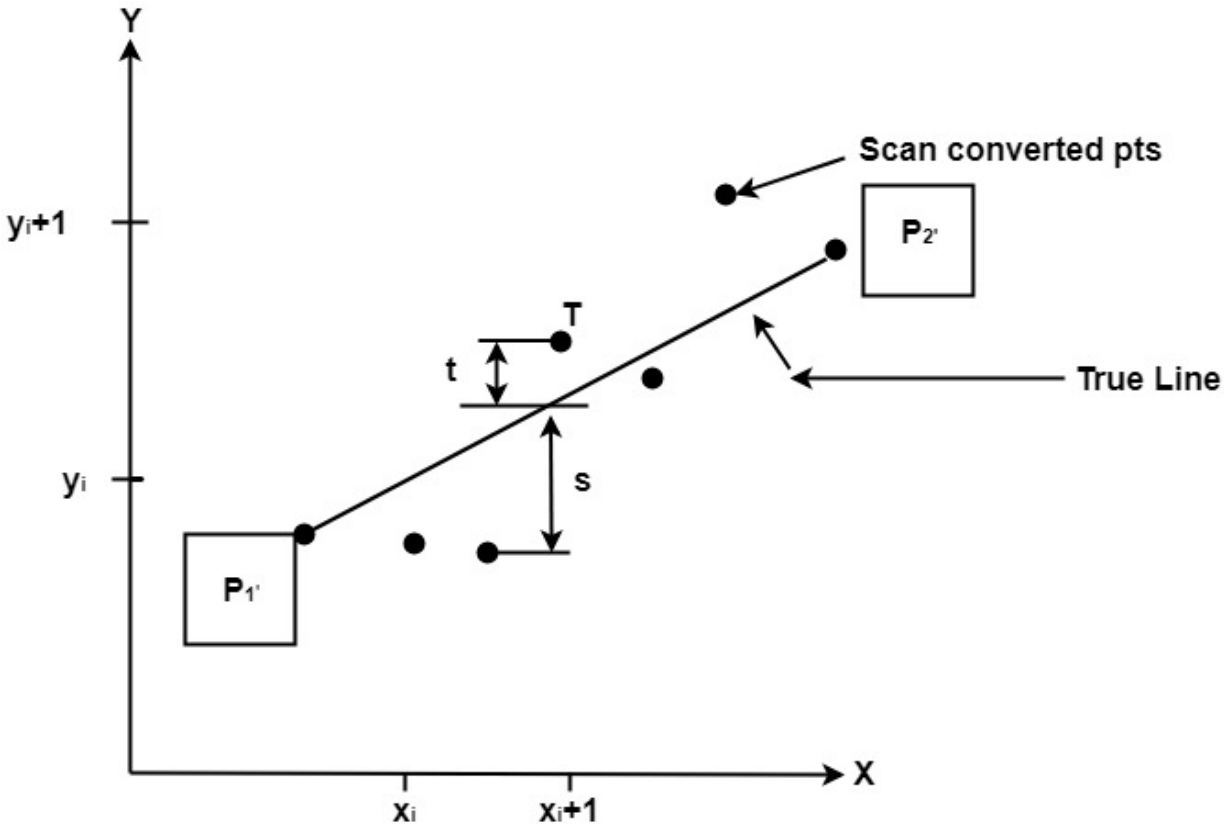


Fig: Scan Converting a line.

To chooses the next one between the bottom pixel S and top pixel T.

If S is chosen

We have $x_{i+1}=x_i+1$ and $y_{i+1}=y_i$

If T is chosen

We have $x_{i+1}=x_i+1$ and $y_{i+1}=y_i+1$

The actual y coordinates of the line at $x = x_{i+1}$ is $y=mx_{i+1}+b$

$$y = m(x_i + 1) + b$$

The distance from S to the actual line in y direction $s = y-y_i$

The distance from T to the actual line in y direction $t = (y_i+1)-y$

Now consider the difference between these 2 distance values s - t

When (s-t) <0 ⟹ s < t

The closest pixel is S

When (s-t) ≥0 ⟹ s < t

The closest pixel is T

This difference is

s-t = (y-yi)-[(yi+1)-y]

= 2y - 2yi -1

$$s - t = 2m(x_i + 1) + 2b - 2y_i - 1$$ [Putting the value of (1)]

Substituting m by $\frac{\Delta y}{\Delta x}$ and introducing decision variable

$d_i=\Delta x$ (s-t)

$d_i=\Delta x$ $(2\frac{\Delta y}{\Delta x}(x_i+1)+2b-2y_i-1)$

$=2\Delta xy_i-2\Delta y-1\Delta x.2b-2y_i\Delta x-\Delta x$

$d_i=2\Delta y.x_i-2\Delta x.y_i+c$

Where c= 2△y+△x (2b-1)

We can write the decision variable d_{i+1} for the next slip on

$d_{i+1}=2\Delta y.x_{i+1}-2\Delta x.y_{i+1}+c$

$d_{i+1}-d_i=2\Delta y.(x_{i+1}-x_i)- 2\Delta x(y_{i+1}-y_i)$

Since x_(i+1)=x_i+1,we have

$d_{i+1}+d_i=2\Delta y.(x_i+1-x_i)- 2\Delta x(y_{i+1}-y_i)$

Special Cases

If chosen pixel is at the top pixel T (i.e., $d_i\geq0$)⟹ $y_{i+1}=y_i+1$

$d_{i+1}=d_i+2\Delta y-2\Delta x$

If chosen pixel is at the bottom pixel T (i.e., $d_i<0$)⟹ $y_{i+1}=y_i$

$d_{i+1}=d_i+2\Delta y$

Finally, we calculate d_1

$d_1=\triangle x[2m(x_1+1)+2b-2y_1-1]$

$d_1=\triangle x[2(mx_1+b-y_1)+2m-1]$

Since $mx_1+b-y_i=0$ and $m = \frac{\triangle y}{\triangle x}$, we have

$d_1=2\triangle y-\triangle x$

Advantage:

1. It involves only integer arithmetic, so it is simple.

2. It avoids the generation of duplicate points.

3. It can be implemented using hardware because it does not use multiplication and division.

4. It is faster as compared to DDA (Digital Differential Analyzer) because it does not involve floating point calculations like DDA Algorithm.

Disadvantage:

1. This algorithm is meant for basic line drawing only Initializing is not a part of Bresenham's line algorithm. So to draw smooth lines, you should want to look into a different algorithm.

Bresenham's Line Algorithm:

Step1: Start Algorithm

Step2: Declare variable $x_1,x_2,y_1,y_2,d,i_1,i_2,dx,dy$

Step3: Enter value of x_1,y_1,x_2,y_2

Where $x_1,y_1$are coordinates of starting point

And x_2,y_2 are coordinates of Ending point

Step4: Calculate $dx = x_2-x_1$

Calculate $dy = y_2-y_1$

Calculate $i_1=2*dy$

Calculate $i_2=2*(dy-dx)$

Calculate $d=i_1-dx$

Step5: Consider (x, y) as starting point and x_{end}as maximum possible value of x.

If $dx < 0$

Then $x = x_2$

$y = y_2$

$x_{end}=x_1$

If dx > 0
Then x = x_1
y = y_1

$x_{end}=x_2$

Step6: Generate point at (x,y)coordinates.

Step7: Check if whole line is generated.
If x > = x_{end}
Stop.

Step8: Calculate co-ordinates of the next pixel
If d < 0
Then d = d + i_1
If d ≥ 0
Then d = d + i_2
Increment y = y + 1

Step9: Increment x = x + 1

Step10: Draw a point of latest (x, y) coordinates

Step11: Go to step 7

Step12: End of Algorithm

x	**y**	**d=d+I_1 or I_2**
1	1	d+I_2=1+(-6)=-5
2	2	d+I_1=-5+8=3
3	2	d+I_2=3+(-6)=-3
4	3	d+I_1=-3+8=5
5	3	d+I_2=5+(-6)=-1
6	4	d+I_1=-1+8=7

7	4	$d+I_2=7+(-6)=1$
8	5	

Example: Starting and Ending position of the line are (1, 1) and (8, 5). Find intermediate points.

Solution: $x_1=1$

$y_1=1$

$x_2=8$

$y_2=5$

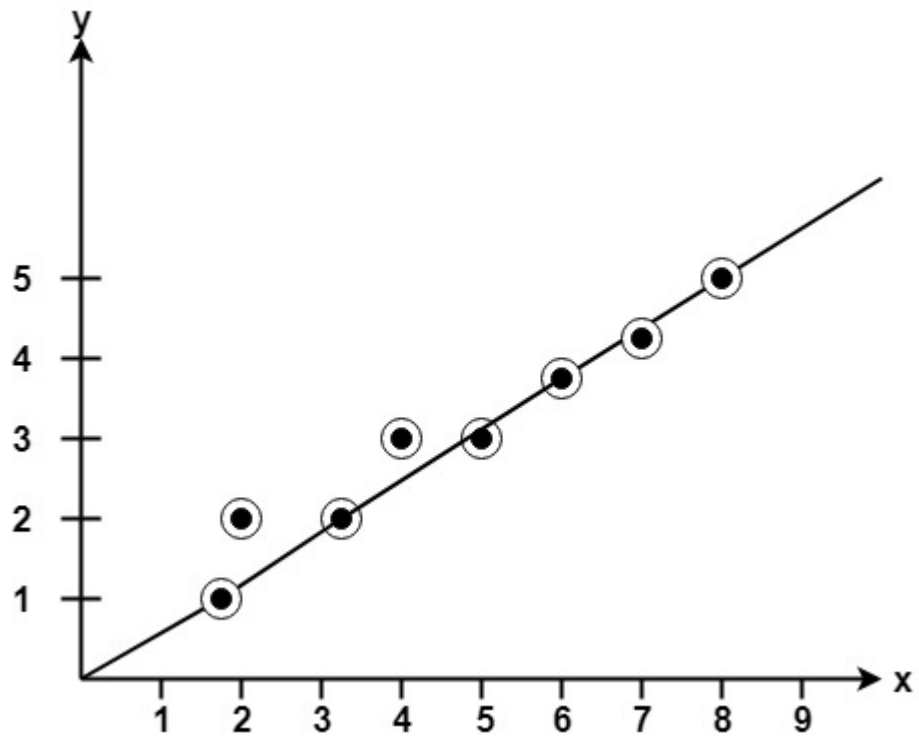
$dx= x_2-x_1=8-1=7$

$dy=y_2-y_1=5-1=4$

$I_1=2* \Delta y=2*4=8$

$I_2=2*(\Delta y-\Delta x)=2*(4-7)=-6$

$d = I_1-\Delta x=8-7=1$



Program to implement Bresenham's Line Drawing Algorithm:

1. #include<stdio.h>
2. #include<graphics.h>
3. **void** drawline(**int** x0, **int** y0, **int** x1, **int** y1)

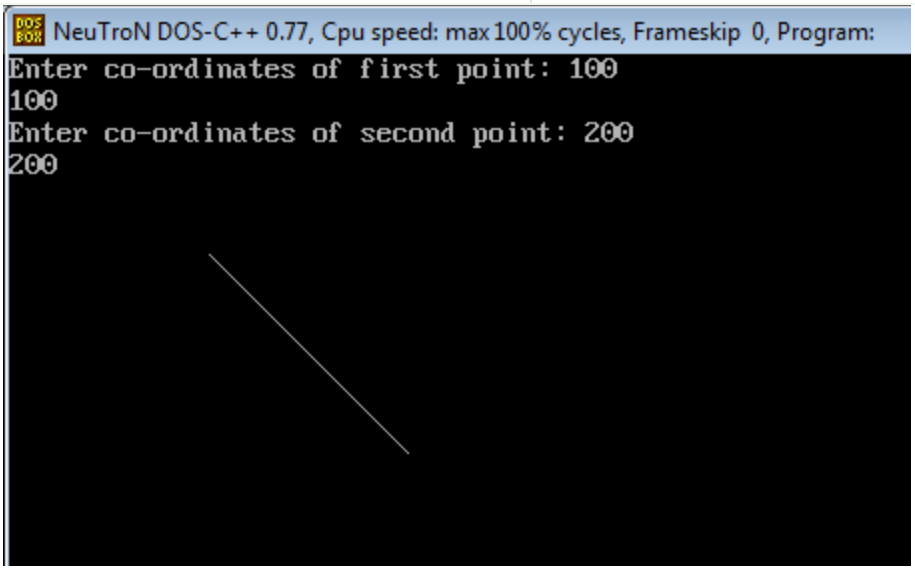
```
{
int dx, dy, p, x, y;
dx=x1-x0;
dy=y1-y0;
x=x0;
y=y0;
p=2*dy-dx;
while(x<x1)
{
if(p>=0)
{
putpixel(x,y,7);
y=y+1;
p=p+2*dy-2*dx;
}
else
{
putpixel(x,y,7);
p=p+2*dy;}
x=x+1;
}
}
int main()
{
int gdriver=DETECT, gmode, error, x0, y0, x1, y1;
initgraph(&gdriver, &gmode, "c:\\turboc3\\bgi");
printf("Enter co-ordinates of first point: ");
```

```
31. scanf("%d%d", &x0, &y0);
32. printf("Enter co-ordinates of second point: ");
33. scanf("%d%d", &x1, &y1);
34. drawline(x0, y0, x1, y1);
35. return 0;
36. }
```

Output:

DDA Algorithm	Bresenham's Line Algorithm
1. DDA Algorithm use floating point, i.e., Real Arithmetic.	1. Bresenham's Line Algorithm use fixed point, i.e., Integer Arithmetic
2. DDA Algorithms uses multiplication & division its operation	2.Bresenham's Line Algorithm uses only subtraction and addition its operation
3. DDA Algorithm is slowly than Bresenham's Line Algorithm in line drawing because it uses real arithmetic (Floating Point operation)	3. Bresenham's Algorithm is faster than DDA Algorithm in line because it involves only addition & subtraction in its calculation and uses only integer arithmetic.
4. DDA Algorithm is not accurate and efficient as Bresenham's Line Algorithm.	4. Bresenham's Line Algorithm is more accurate and efficient at DDA Algorithm.

5.DDA Algorithm can draw circle and curves but are not accurate as Bresenham's Line Algorithm	5. Bresenham's Line Algorithm can draw circle and curves with more accurate than DDA Algorithm.



Differentiate between DDA Algorithm and Bresenham's Line Algorithm: